



UNIVERSIDAD DE LAS FUERZAS ARMADAS - ESPE

DEPARTAMENTO DE CIENCIAS DE LA
COMPUTACIÓN

ASEGURAMIENTO DE LA CALIDAD - 30733

Consulta Grupal - Modelo de Boehm

Docente:

Ing. David Gamboa

Estudiante:

Kevin Asmal, Fernando Tipán, Mateo Llumigusín, Gabriel López, Marcelo Pareja

29 de abril de 2026

1. Introducción

En el desarrollo de software, garantizar la calidad del producto final es un aspecto fundamental para satisfacer las necesidades de los usuarios y asegurar su correcto funcionamiento en diferentes entornos. Se han propuesto diversos modelos de evaluación, entre los cuales, el modelo de calidad de Boehm, organiza características del software en diferentes niveles o capas, permitiendo analizar tanto aspectos generales como detalles técnicos y específicos.

Este trabajo presenta una descripción estructurada de dichas características, clasificándolas en niveles de alto nivel, intermedio y primitivo, con el objetivo de comprender cómo cada una contribuye a la calidad global del software. Además, se analiza la relación entre estas capas y su importancia en la evaluación del producto.

2. Desarrollo

2.1. Definición

El modelo jerárquico de calidad de software fue propuesto por Barry Boehm en 1978, durante una etapa conocida como la “crisis del software”. En la década de 1970, el uso de sistemas informáticos crecía rápidamente en empresas, gobiernos y organizaciones, pero muchos proyectos presentaban errores, retrasos y altos costos de desarrollo. Esto generó la necesidad de crear métodos que permitieran evaluar y mejorar la calidad del software de manera más organizada.

La motivación principal de Boehm fue establecer un modelo que ayudaría a medir la calidad no solo desde el funcionamiento técnico del sistema, sino también considerando aspectos como la confiabilidad, eficiencia, facilidad de uso y mantenimiento. Su propuesta buscaba garantizar que el software cumpliera las necesidades reales de los usuarios y pudiera mantenerse correctamente con el paso del tiempo.

El entorno tecnológico de esa época era muy diferente al actual. Los sistemas se desarrollaban principalmente para computadoras centrales (mainframes), con recursos limitados de hardware y procesos de programación secuenciales. Además, las metodologías ágiles y las prácticas modernas como DevOps aún no existían, por lo que la calidad se evaluaba principalmente al final del desarrollo. En este contexto, el modelo de Boehm representó un avance importante al introducir una visión estructurada y jerárquica de la calidad del software.

2.2. Características por capas del modelo

2.2.1. Características de Alto nivel

Las características de alto nivel, se conocen también como “*objetivos globales de calidad*”, ya que son los criterios que usan los usuarios finales para evaluar la calidad de un producto de software. Como indica Boehm, al momento de adquirir un producto de software, se suelen formular tres preguntas:

1. ¿Qué tan útil es?

2. ¿Qué tan fácil es de mantener?
3. ¿Se puede usar en un entorno diferente?

Cada una de estas preguntas está relacionada a una característica de alto nivel.

- **Utilidad:** Hace referencia al grado en que el software responde y satisface las necesidades del usuario. En inglés, se usa el término de utilidad “as-is”, lo que indica que el software debe ser capaz de ser usado con facilidad y cumplir las funciones planificadas en su *estado original*, sin necesidad de cambios o mejoras.
- **Mantenibilidad:** Representa el esfuerzo o dificultad necesaria para realizar cambios en el software tras su lanzamiento, sean actualizaciones, mejoras o correcciones.
- **Portabilidad:** Es la capacidad del software para ser adaptado a diferentes entornos: sistemas operativos, dispositivos y tecnologías, sin perder su funcionalidad.

2.2.2. Características de Nivel Intermedio

El nivel intermedio conecta las características de alto nivel con las primitivas (más específicas y técnicas). Sus atributos principales son:

- **Portabilidad:** Capacidad del software para ser transferido y ejecutado en diferentes entornos o plataformas sin grandes modificaciones.
- **Fiabilidad:** Probabilidad de que el sistema funcione correctamente bajo condiciones específicas durante un tiempo determinado.
- **Eficiencia:** Uso óptimo de recursos como memoria, tiempo de ejecución y capacidad de procesamiento.
- **Usabilidad:** Facilidad con la que los usuarios pueden aprender, entender y operar el sistema.
- **Capacidad de prueba (testabilidad):** Facilidad para diseñar y ejecutar pruebas que validen el correcto funcionamiento del software.
- **Comprensión (understandability):** Qué tan fácil es para los desarrolladores entender el código y la estructura del sistema.
- **Flexibilidad:** Posibilidad de modificar el software para adaptarlo a nuevas necesidades o requisitos.

2.2.3. Características de Nivel primitivo

Son características que se asocian a una o dos métricas de calidad, y a partir de ellas se crean factores medibles con los que calificar al software. Entre estos están agrupados por las características de nivel intermedio:

- **Portabilidad**

- *Independencia de dispositivos*: Capacidad del software de funcionar en cualquier dispositivo
- *Autocontención*

■ **Confiabilidad**

- *Compleitud* Todos los módulos de software están en correcto funcionamiento
- *Consistencia* Cada módulo devuelve y procesa datos precisos y uniformes durante el funcionamiento del sistema.
- *Robustez/Integridad*
- *Auto-contención de confiabilidad* Capacidad del software de funcionar autónomamente
- *Exactitud* Capacidad de generar respuestas similares al esperado

■ **Eficiencia**

- *Accesibilidad* Capacidad tanto de permitir el uso por varios tipos de usuarios y mantengan un funcionamiento correcto
- *Eficiencia de uso de dispositivo* Sin depender del dispositivo el software debe procesarse correctamente y sin bajar su rendimiento.

■ **Facilidad de pruebas**

- *Comunicación* Comunicación efectiva entre módulos y tecnologías, con tal de probarlos
- *Auto descripción* Cada módulo, función y prueba muestra que realizará y los resultados de cada uno
- *Estructuración* La estructura permite realizar pruebas y mostrar los resultados esperados

■ **Facilidad de Comprensión**

- *Consistencia* La información dada por el sistema es tanto correcta como legible por los usuarios
- *Estructuración* Con bases concretas, el sistema cuenta con módulos y presentación entendibles a usuarios
- *Conciso* El sistema esta conformados por módulos que no desperdician recursos y mantienen funcionalidades con lo mínimo necesario
- *Legibilidad* Es comprensible tanto módulos y funciones como resultados proporcionados a los usuarios

■ **Facilidad de modificación**

- *Estructuración* La organización permite al software evolucionar y ser resilientes a cambios

- *Aumentividad* Se pueden generar nuevos módulos manteniendo funcionalidades anteriores

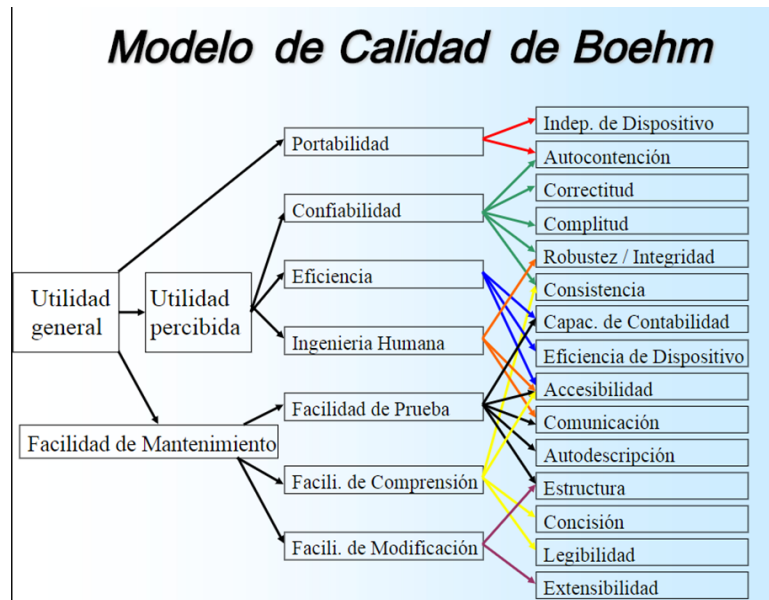


Figura 1: Relación entre componentes por capas

2.3. Ventajas del modelo

El modelo de calidad propuesto por Barry Boehm presenta diversas ventajas que lo convierten en una herramienta sólida para la evaluación del software, ya que permite analizar de manera integral tanto las necesidades del usuario como los aspectos técnicos del sistema.

- Permite una **evaluación estructurada**, ya que organiza la calidad del software en niveles (alto, intermedio y primitivo).
- Facilita la **comprensión del software**, al relacionar aspectos generales con detalles técnicos más específicos.
- Está **orientado al usuario**, considerando la utilidad, mantenibilidad y portabilidad como factores clave.
- Proporciona una **visión integral de la calidad**, incluyendo factores como eficiencia, usabilidad, fiabilidad y flexibilidad.
- Permite **medir la calidad**, gracias a sus características primitivas que se pueden asociar a métricas.
- Es flexible y adaptable a diferentes tipos de proyectos y etapas de desarrollo [11].
- Ayuda a **identificar fallos y debilidades** en el software de manera más precisa.
- Favorece la **mejora continua**, permitiendo realizar cambios, pruebas y mantenimiento con mayor facilidad.

- Es **flexible y adaptable**, ya que puede aplicarse a diferentes tipos de sistemas y proyectos.
- Apoya la **toma de decisiones**, al ofrecer criterios claros para evaluar y comparar productos de software.
- Puede utilizarse en **distintas etapas del desarrollo**, desde el diseño hasta la evaluación final.

2.4. Desventajas del modelo

El modelo de Boehm se limita bastante por la cantidad de características a tomar en cuenta, no define en cierta medida como debe cuantificarse ni tampoco expone en su totalidad como cada característica es utilizada

- No se explica en muchos casos el cómo debe escogerse entre diferentes componentes o cómo priorizar una característica sobre otra.
- Con la gran cantidad de características a evaluar, se vuelve muy complejo probar cada una y es difícil definir la calidad en cada uno.
- No expone como realmente calcular la métrica medible o un valor para las características de eficiencia, exactitud, etc.
- Muchos de los modelos se centran en el lado del programador que del usuario, a pesar de tomar en cuenta accesibilidad y usabilidad, no detalla a la retroalimentación de los usuarios como una parte fundamental del modelo.
- Los niveles intermedios de Boehm no son los más detallados a comparación de los niveles altos y primitivos que al ser conceptos conocidos, el modelo jerárquico no ayuda a comprender tanto las relaciones como su utilidad en el modelo.
- En la actualidad, aunque los conceptos de calidad fueron muy bien diseñados, eran para sistemas de los años 70, no toma en cuenta factores como seguridad, interoperabilidad, microservicios, sincronía entre otras tecnologías modernas.

2.5. Comparación con otros modelos de calidad

El modelo de Boehm es muy cercano al modelo McCall ya que mantiene la jerarquía y el uso de características para evaluar la calidad de un producto. Sin embargo, considera en mayor medida al usuario y sus necesidades, ya que incluye el atributo de ingeniería humana que no estaba presente en el modelo de McCall, y coloca a las características que los usuarios perciben en el primer nivel. Esto permite establecer un mejor balance entre la calidad desde las perspectivas de clientes y desarrolladores.

Por otra parte, si se compara con modelos más recientes, presenta algunas carencias. El modelo de Boehm es una de las principales influencias en la creación de los estándares ISO de calidad de software, incluido el 25010, ya que pasa las características del nivel medio a las características

principales del ISO. Sin embargo, no toma en cuenta aspectos como la seguridad, que es crucial hoy en día, o la usabilidad, ya que si bien considera la perspectiva del usuario aún maneja atributos muy técnicos para medir su calidad en ese aspecto.

2.6. Escenario Hipotético

Una Fintech está migrando su sistema monolítico a una arquitectura de microservicios basada en la nube. El objetivo es asegurar que el nuevo sistema no solo funcione hoy (Utilidad Inicial), sino que sea fácil de mantener y migrar en el futuro (Utilidad a Largo Plazo), siguiendo la visión de Boehm.

Aplicación de la Estructura Jerárquica: Para priorizar los atributos de calidad, aplicamos el modelo de arriba hacia abajo (enfoque descendente):

2.6.1. Nivel 1 (Utilidad Global)

El sistema debe procesar transacciones sin errores y permitir actualizaciones frecuentes sin interrumpir el servicio. Si el sistema no es "útil" bajo estas condiciones, el proyecto fracasa.

2.6.2. Nivel 2 y 3 (Factores de Calidad)

Seleccionamos tres de los siete factores de Boehm para este caso:

■ Mantenibilidad (Modificabilidad)

- Criterio: Estructurabilidad; Los microservicios deben estar desacoplados.
- Acción: Evaluar si un cambio en el servicio de Conversión de Moneda afecta al servicio de Auditoría.

■ Mantenibilidad (Modificabilidad)

- Criterio: Independencia del Entorno.
- Acción: El software debe poder moverse de un proveedor de nube (AWS) a otro (Azure) o a servidores locales si las regulaciones cambian.

■ Mantenibilidad (Modificabilidad)

- Criterio: Precisión
- Acción: Los cálculos de tasas e impuestos deben ser exactos según la normativa de cada país.

2.6.3. Nivel 4: Métricas Cuantitativas (Evaluación)

Para que el modelo de Boehm sea efectivo, definimos métricas concretas:

Factor	Métrica Sugerida	Meta
Mantenibilidad	Tiempo medio de reparación (MTTR)	menor a 2 horas
Portabilidad	% de código dependiente del sistema operativo	0 % (Uso de Docker)
Fiabilidad	Tasa de éxito de transacciones	99.99 %
Eficiencia	Tiempo de respuesta bajo carga máxima	Menor a 200ms

Tabla 1: Análisis de los atributos con métricas de calidad

2.6.4. Aplicación Práctica en el Ciclo de Vida

Durante la fase de diseño, el equipo utiliza el modelo de Boehm para tomar una decisión crítica:

- **Dilema:** ¿Usar una base de datos propietaria muy rápida (Eficiencia) o una estándar compatible con múltiples sistemas (Portabilidad)?
- **Decisión bajo Boehm:** El modelo de Boehm otorga un peso significativo a la "Utilidad a largo plazo". El equipo decide sacrificar un 5 % de rendimiento por una mayor portabilidad, asegurando que el software pueda evolucionar y sobrevivir a cambios de infraestructura tecnológica, evitando el "vendor lock-in".

2.6.5. Resultados del Análisis

1. **Priorización Clara:** Los desarrolladores saben que la legibilidad del código es más importante que la optimización extrema del CPU.
2. **Trazabilidad:** Si una métrica falla (Nivel 4), se identifica inmediatamente qué factor de calidad (Nivel 2) está en riesgo y cómo afecta a la utilidad global (Nivel 1).

3. Conclusiones

El modelo de Boehm es un modelo jerárquico con el fin de mejorar las falencias del modelo de McClall para la calidad de software, clasifica los atributos de calidad por capas o niveles según su cercanía a una perspectiva de usuario o una perspectiva más técnica, no segmenta los principios, sino que define relaciones entre características y métricas de calidad que existen. Este enfoque permite realizar pruebas y correcciones especializadas en ciertos atributos para aumentar el nivel general de calidad, siendo tomado como referencia para otros estándares. Sin embargo, se debe aplicar de forma correcta ya que la estructura de atributos no es completamente sólida y al intentar cumplir objetivos de calidad muy específicos se pueden descuidar otros aspectos.

4. Bibliografía

1. Boehm, B. W., Brown, J. R., & Lipow, M. (1976). Quantitative evaluation of software quality. *Software Engineering: Barry W. Boehm's Lifetime Contributions to Software Development, Management, and Research*, 25.
2. Cano, M. A. (2025, octubre 7). Modelo de calidad del software de Boehm. *Prezi*. <https://prezi.com/p/hk5exv3xdy4s/modelo-de-calidad-del-software-de-boehm/>
3. Modelos Eredigitales. (2018, julio). El modelo de Boehm. *Blogspot*. <https://modeloseredigitales.blogspot.com/2018/07/el-modelo-de-boehm.html>
4. Professional QA. (s.f.). Boehm software quality model. <https://www.professionalqa.com/boehm-software-quality-model>
5. Santhiya, S. (s.f.). *SQM unit 1 and unit 2* [Presentación de diapositivas]. Scribd. <https://www.scribd.com/presentation/64046141/SQM-Unit1-and-Unit-2>
6. Modelos de evaluación de recursos educativos digitales. (s.f.). *Blogspot*. Recuperado el 29 de abril de 2026, de <https://modeloseredigitales.blogspot.com/2018/07/el-modelo-de-boehm.html>
7. Modelos de Evaluación RED Grupo 9. (s.f.). Modelo de evaluación Boehm. *Fandom Wiki*. https://modelos-de-evaluacion-red-grupo9.fandom.com/wiki/MODELO_DE_EVALUACION_BOEHM